

Classification Algorithms Report

Classification is a supervised machine learning technique used to predict labels or categories from input data. It assigns each data point to a predefined class based on learned patterns.

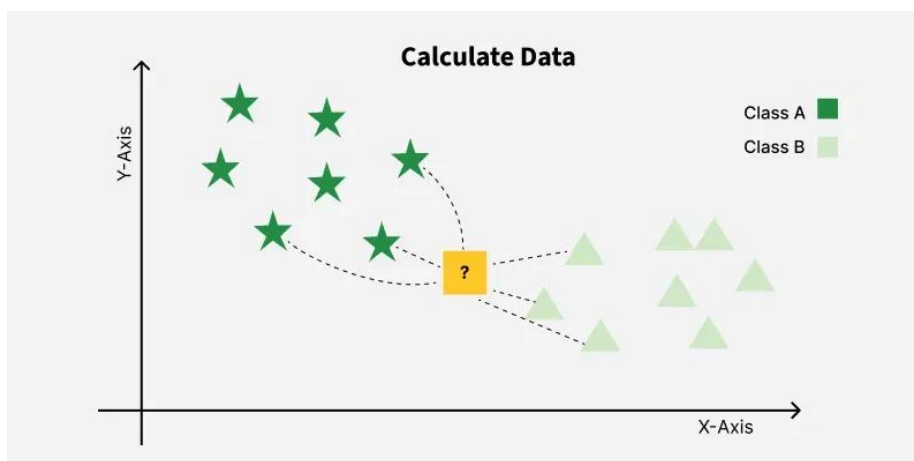
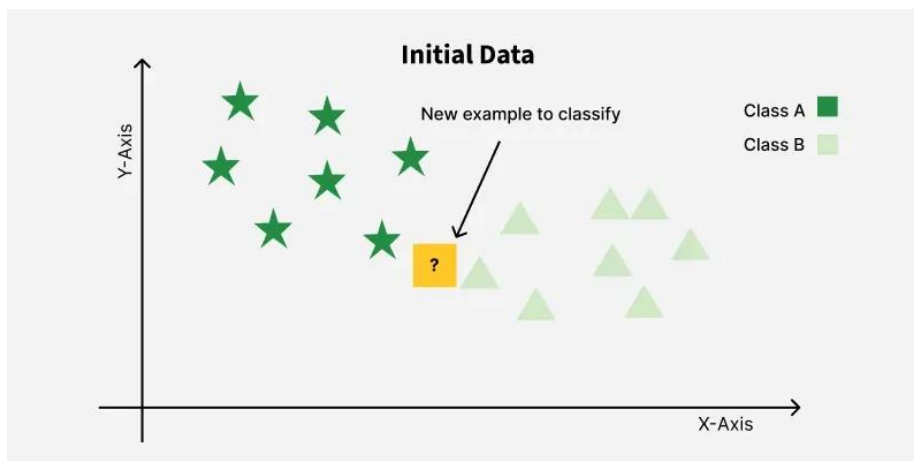
In this classification report, algorithms such as KNN (K-Nearest Neighbour), Naive Bayes, Logistic Regression, Random Forest and SVM (Support Vector Machine) is implemented. Each algorithm is explained theoretically and with Python programs.

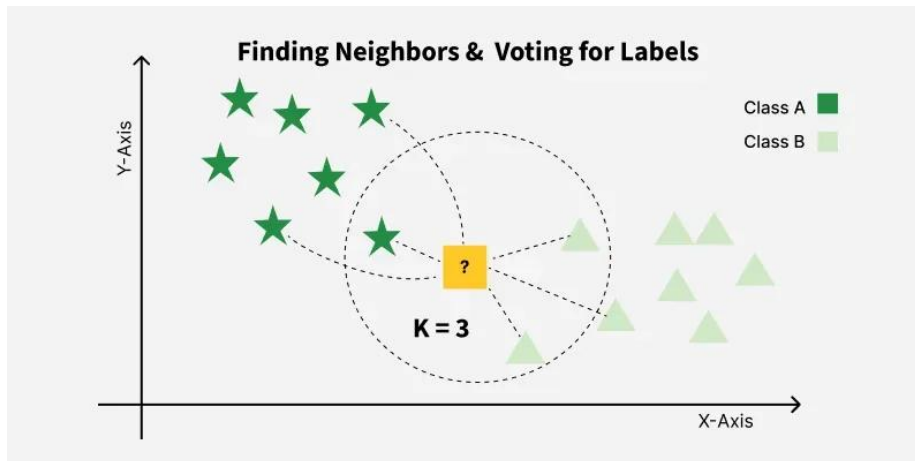
1.KNN (K-Nearest Neighbours)

K-Nearest Neighbor (KNN) is a simple and widely used machine learning technique for classification and regression tasks. It works by identifying the K closest data points to a given input and making predictions based on the majority class or average value of those neighbors.

It uses Euclidean Distance to find the distance between its neighbours.

KNN makes no assumptions about the underlying data distribution, it makes it a non-parametric and instance based learning method.





As it can be seen from the images, K is a number that tells the algorithm how many nearby points or neighbors to look at when it makes a decision.

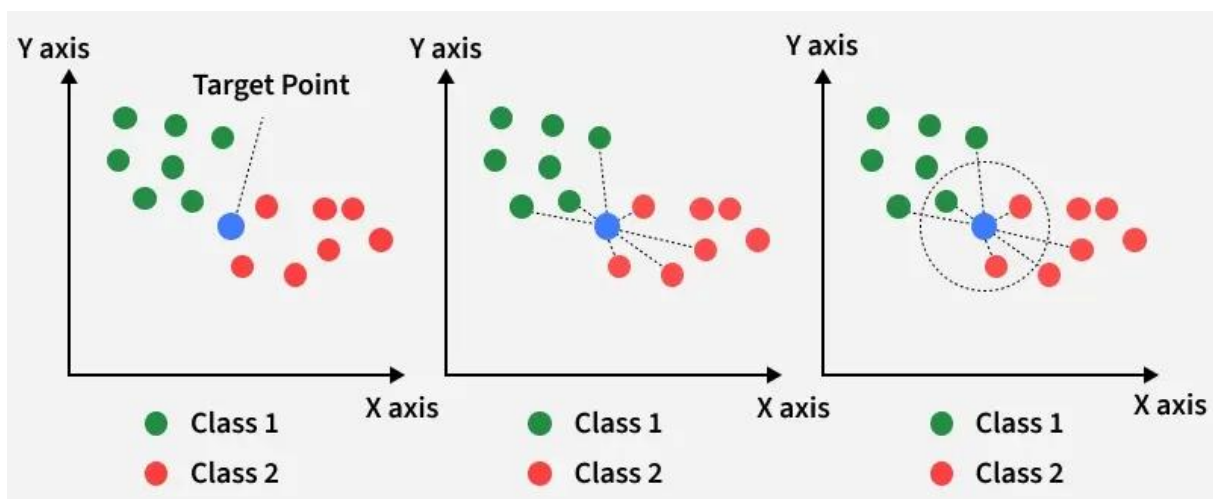
Since we give $k=3$ here, the algorithm will take 3 neighbours of the given point.

By looking at the point, we can clearly see the point has 2 class B neighbours and 1 class A neighbour. Since class B is bigger, the point will be predicted as class B.

If a data point has too many outliers, using a larger k can make the predictions more stable.

But if k is too large the model may become too simple and miss important patterns and this is called underfitting.

So k value must be carefully chosen.



1.2 KNN Python Example

In this example, KNN will be implemented using scikit-learn library. Here is all the libraries we used for this program.

```
1  from sklearn.datasets import make_moons
2  import matplotlib.pyplot as plt
3  import seaborn as sns
4  import pandas as pd
5  from sklearn.preprocessing import StandardScaler
6  from sklearn.model_selection import train_test_split, cross_val_score
7  from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixDisplay, confusion_matrix
8  from sklearn.neighbors import KNeighborsClassifier
9  import numpy as np
```

From sklearn, we imported functions such as make_moons, StandardScaler, train_test_split, cross_val_score and metrics functions.

Make_moons creates the artificial moon-shaped dataset. It is useful for demonstrating how KNN handles nonlinear class boundaries.

StandardScaler standardize features by removing the mean and scaling to unit variance.

Matplotlib is used to create plot graphs and show dataset created by using make_moons.

Seaborn library provides higher-level statistical visualizations.

From model selection, train_test_split is used to train the data and cross_val_score is used for performing cross-validation.

From metrics, we imported many extensions that explained later with their outputs.

From sklearn.neighbors, KNeighborsClassifier is imported to implement KNN.

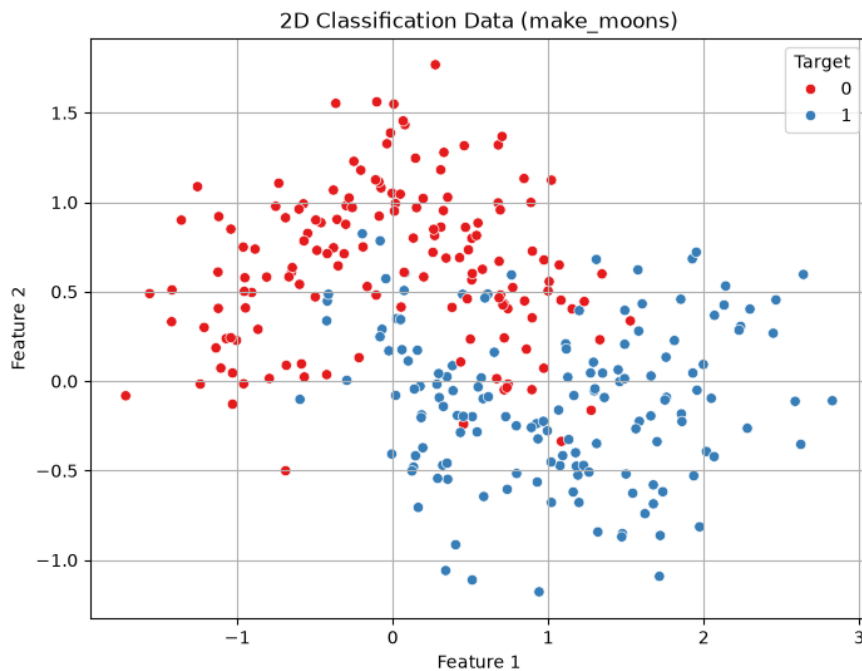
Numpy is used mostly for numerical operations.

```
10
11  X, y = make_moons(n_samples=300, noise=0.3, random_state=42)
12
13  df = pd.DataFrame(X, columns=["Feature 1", "Feature 2"])
14  df['Target'] = y
15
16  plt.figure(figsize=(8,6))
17  sns.scatterplot(data=df, x="Feature 1", y="Feature 2", hue="Target", palette="Set1")
18  plt.title("2D Classification Data (make_moons)")
19  plt.grid(True)
20  plt.savefig("2D_Classification_Data.png")
21  plt.show()
```

Here, we used make_moons to create synthetic dataset that has 300 samples.

X is features and y is target of the dataset.

Matplotlib used to visualize the dataset. Here is the output:



To implement KNN, we must know the categories of the each data point.

```
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.3, random_state=42, stratify=y)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Here, we decided %30 of the dataset for testing and %70 percent for training.

StandardScaler is implemented to standardize the features.

```
28 knn = KNeighborsClassifier(n_neighbors=5)
29 knn.fit(X_train, y_train)
30 y_pred = knn.predict(X_test)
31 print(f"Test Accuracy (k=5): {accuracy_score(y_test, y_pred):.2f}")
```

We created the model by defining k=5. After that model is trained and predictions are implemented.

Accuracy score is printed out. Here is the output:

```
Test Accuracy (k=5): 0.89
```

```
k_range = range(1, 21)
cv_scores = []

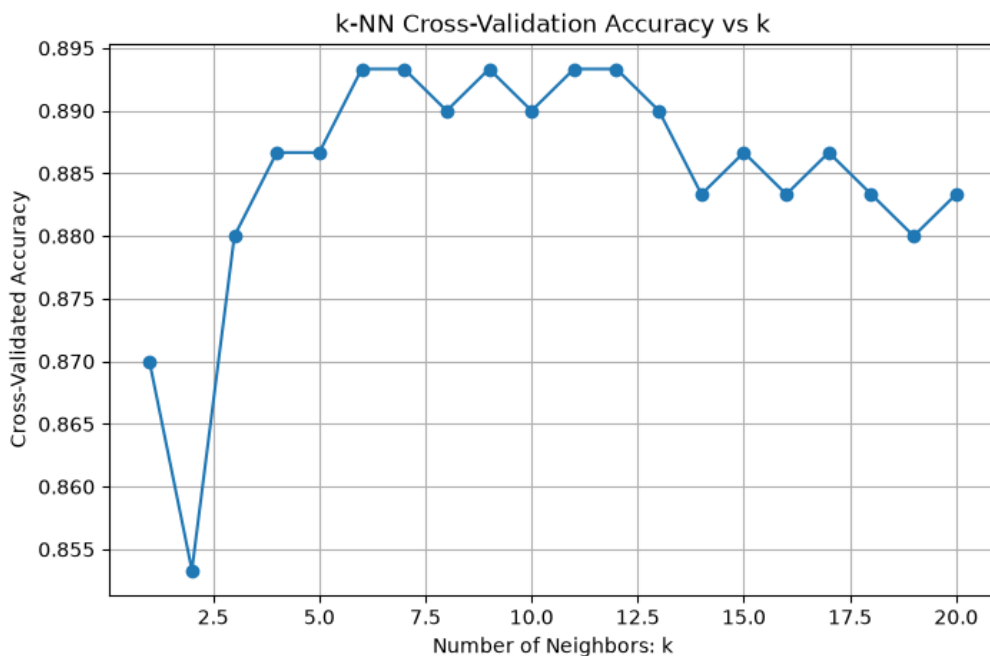
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_scaled, y, cv=5, scoring="accuracy")
    cv_scores.append(scores.mean())

plt.figure(figsize=(8,5))
plt.plot(*args: k_range, cv_scores, marker='o')
plt.title("k-NN Cross-Validation Accuracy vs k")
plt.xlabel("Number of Neighbors: k")
plt.ylabel("Cross-Validated Accuracy")
plt.grid(True)
plt.savefig("k-NN_Cross-Validation_Accuracy.png")
plt.show()
```

K_range is used inside for loop to find accuracy score of each k value from 1 to 21.

Cv_scores is used to store each k values accuracy.

Matplotlib is used to visualize this. Here is the output:



From the graph we can see some of the values are really accurate and some of them are not.

```
best_k = k_range[np.argmax(cv_scores)]  
print(f"Best k from cross-validation: {best_k}")  
print("\n")
```

Here, we found the best k value for accuracy which is 6.

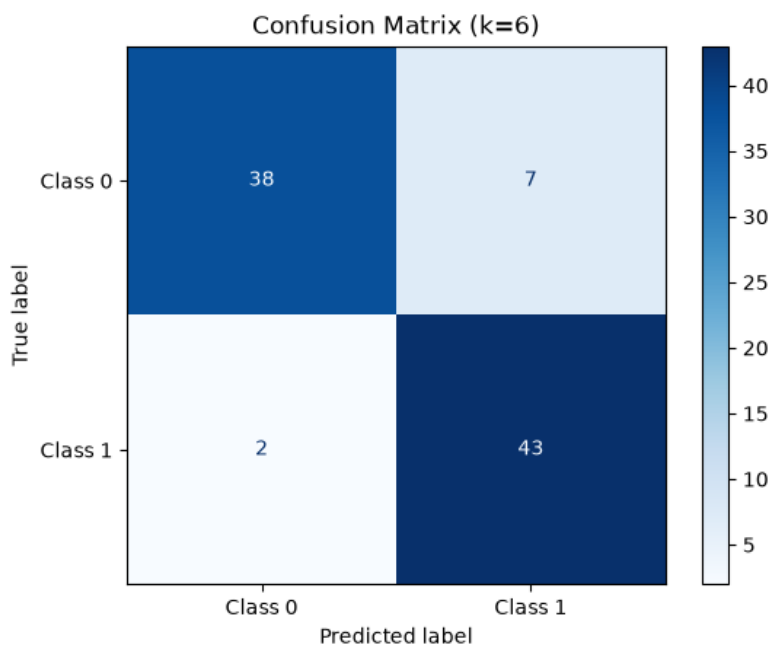
```
Best k from cross-validation: 6
```

```
best_knn = KNeighborsClassifier(n_neighbors=best_k)  
best_knn.fit(X_train, y_train)  
y_pred = best_knn.predict(X_test)
```

We implemented another knn model with k=6.

```
cm = confusion_matrix(y_test, y_pred)  
display = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Class 0", "Class 1"])  
  
display.plot(cmap="Blues")  
plt.title(f"Confusion Matrix (k={best_k})")  
plt.grid(False)  
plt.savefig("Confusion_Matrix.png")  
plt.show()
```

This part creates a confusion matrix with the value k=6. Here is the output:



As we can clearly see, model is pretty accurate. Only 7 values in class 0 and 2 values in class 1 are wrongly defined.

```
print("Classification Report")
print(classification_report(y_test, y_pred, target_names=["Class 0", "Class 1"]))
```

This part prints out the classification report. Here is the output:

Classification Report				
	precision	recall	f1-score	support
Class 0	0.95	0.84	0.89	45
Class 1	0.86	0.96	0.91	45
accuracy			0.90	90
macro avg	0.91	0.90	0.90	90
weighted avg	0.91	0.90	0.90	90

```
X_min, X_max = X_scaled[:, 0].min() - 1, X_scaled[:, 0].max() + 1
y_min, y_max = X_scaled[:, 1].min() - 1, X_scaled[:, 1].max() + 1

xx, yy = np.meshgrid(np.linspace(X_min, X_max, num=300), np.linspace(y_min, y_max, num=300))
Z = best_knn.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

This part creates a dense grid covering the entire feature space and predicts the class at every point. These predictions are later used to display the KNN decision boundary.

Numpy np here is used to create 300 equally spaced feature1 and 2.

```
plt.figure(figsize=(8,6))
plt.contourf(*args: xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.3)
sns.scatterplot(x=X_scaled[:,0], y=X_scaled[:,1], hue=y, palette="Set1", edgecolor="k")
plt.title(f"Decision Boundary with Best k = {best_k}")
plt.xlabel("Feature 1 (scaled)")
plt.ylabel("Feature 2 (scaled)")
plt.grid(True)
plt.savefig("Decision_Boundary_with_Best_k.png")
plt.show()
```

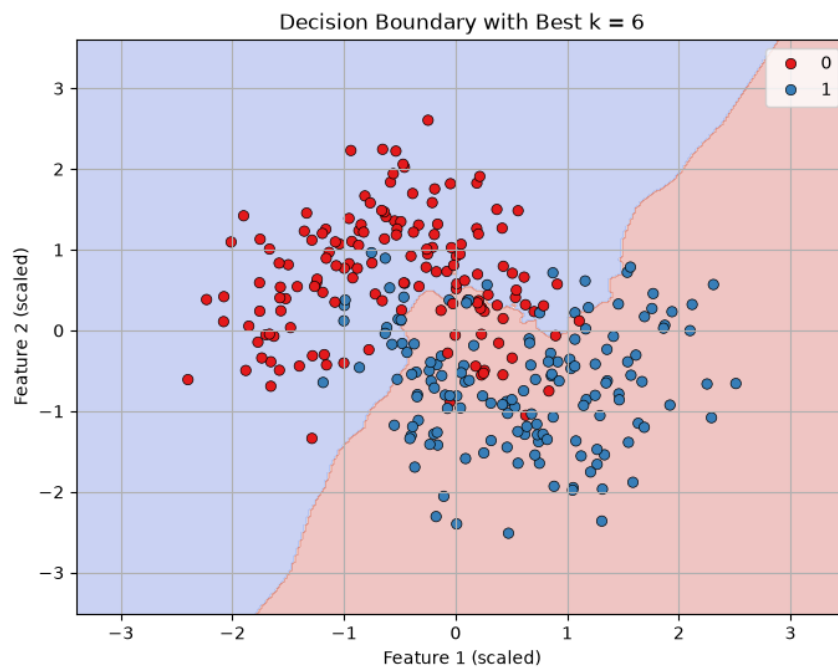
Lastly, we used matplotlib to create the figure and seaborn to scatter the points.

Plt.contourf creates the colored decision regions.

Cmap coolwarm assigns contrasting colors to the classes, normally blue and red.

Decision Boundary at the next page:

Classification Algorithms Report



We can see that our KNN model is working fine in classifying datapoints.

1.3 KNN Program with Real Dataset

In this program, we used an Banknote Authentication dataset from UCI Machine Learning Repository. Data were extracted from images that were taken for the evaluation of an authentication procedure for banknotes.

Dataset has no missing values, its fully numerical and there is class column which has only 1 and 0 binary outputs. Here is some part from the dataset.

	variance,skewness,curtosis,entropy,class
1	4.5459,8.1674,-2.4586,-1.4621,0
2	3.866,-2.6383,1.9242,0.10645,0
3	3.4566,9.5228,-4.0112,-3.5944,0
4	0.32924,-4.4552,4.5718,-0.9888,0
5	4.3684,9.6718,-3.9606,-3.1625,0
6	3.5912,3.0129,0.72888,0.56421,0
7	2.0922,-6.81,8.4636,-0.60216,0
8	3.2032,5.7588,-0.75345,-0.61251,0
9	1.5356,9.1772,-2.2718,-0.73535,0
10	1.2247,8.7779,-2.2135,-0.80647,0
11	3.9899,-2.7066,2.3946,0.86291,0
12	1.8993,7.6625,0.15394,-3.1108,0
13	-1.5768,10.843,2.5462,-2.9362,0
14	3.404,8.7261,-2.9915,-0.57242,0
15	4.6765,-3.3895,3.4896,1.4771,0
16	2.6719,3.0646,0.37158,0.58619,0
17	0.80355,2.8473,4.3439,0.6017,0
18	1.4479,-4.8794,8.3428,-2.1086,0
19	5.2423,11.0272,-4.353,-4.1013,0
20	

Here is all the libraries imported in the program.

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixDisplay, confusion_matrix
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.utils.fixes import parse_version
import scipy.stats
import numpy as np
```

Pandas is imported for reading and printing the dataset information.

Same sklearn extensions are implemented from previous KNN example.

Since the dataset is txt file, scipy.stats and sklearn.parse_version is implemented to fix errors.

Numpy and matplotlib is also added for the program which have the same function from the previous program.

```
pd.set_option(*args: 'display.max_columns', 15)
pd.set_option(*args: 'display.width', 350)
pd.set_option(*args: 'display.max_rows', 100)
pd.set_option(*args: 'display.max_colwidth', 100)

df = pd.read_csv('data_banknote_authentication.txt')

print("\nFirst 5 Rows of the Dataset")
print(df.head())

print("\nLast 5 Rows of the Dataset")
print(df.tail())

print("\nDataset Shape")
print(df.shape)

print("\nDataset Information")
print(df.info())

print("\nDataset Description")
print(df.describe())

print("\nMissing Values in the Dataset")
print(df.isnull().sum())
print("There are no missing values in the Dataset\n")
```

Dataset information is printed out.

```
X = df[["variance", "skewness", "curtosis", "entropy"]]
y = df["class"]

X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.3, random_state=23, stratify=y)
```

Features(X) and target(y) is defined.

%30 of the data is used for testing and %70 of it used for training.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=4)
knn.fit(X_train_scaled, y_train)
y_pred = knn.predict(X_test_scaled)
```

StandardScaler is implemented on features.

KNN model is trained and data predicted with k=4.

```
print("Accuracy Score")
print(accuracy_score(y_test, y_pred))
print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

Accuracy score and classification report is printed out. Here is the output:

```
Accuracy Score
0.9975728155339806

Classification Report
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	229
1	0.99	1.00	1.00	183
accuracy			1.00	412
macro avg	1.00	1.00	1.00	412
weighted avg	1.00	1.00	1.00	412

It can clearly seen that model is pretty accurate.

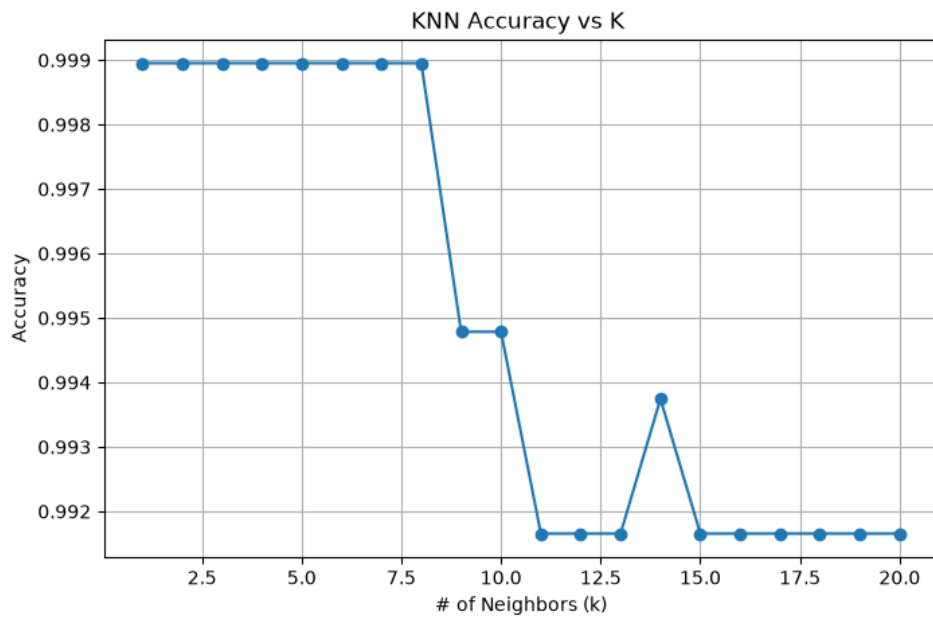
```
k_range = range(1, 21)
cv_scores = []

for k in k_range:
    model = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=5, scoring="accuracy")
    cv_scores.append(scores.mean())

plt.figure(figsize=(8, 5))
plt.plot(*args: k_range, cv_scores, marker='o')
plt.title("KNN Accuracy vs K ")
plt.xlabel("# of Neighbors (k)")
plt.ylabel("Accuracy")
plt.grid(True)
plt.savefig("Accuracy_of_each_k_value.png")
plt.show()
```

This part is mostly same with previous KNN example. We found each k values accuracy and visualize using matplotlib.

Graph at the next page:



As it can be seen from the graph k values from 1 to 8 are pretty accurate, nearly 100 percent.

```
best_k = k_range[np.argmax(cv_scores)]
print(f"Best k from cross-validation: {best_k}")
print("\n")

best_knn = KNeighborsClassifier(n_neighbors=best_k)
best_knn.fit(X_train_scaled, y_train)
y_pred = best_knn.predict(X_test_scaled)
```

Just like the previous KNN example, most accurate k value is found and model is trained using this value which is k=1.

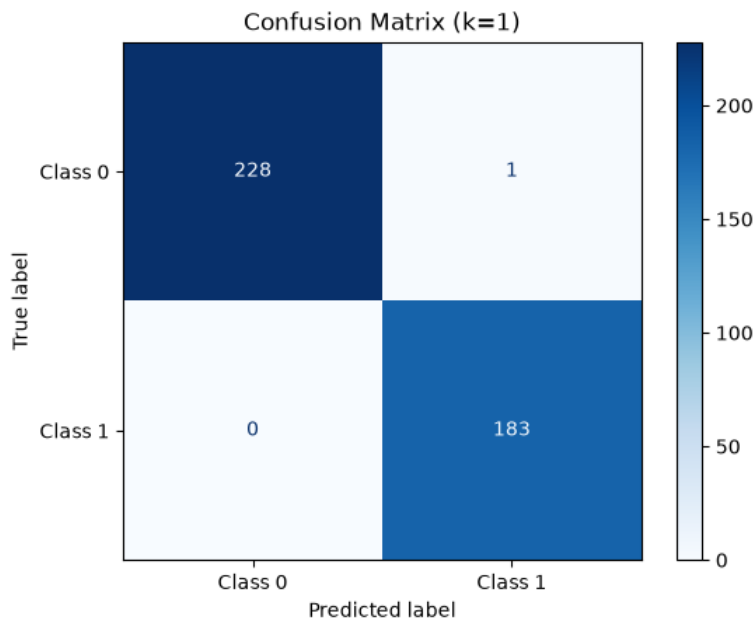
```
Best k from cross-validation: 1
```

```
cm = confusion_matrix(y_test, y_pred)
display = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Class 0", "Class 1"])

display.plot(cmap="Blues")
plt.title(f"Confusion Matrix (k={best_k})")
plt.grid(False)
plt.savefig("Confusion_Matrix.png")
plt.show()
```

Confusion matrix is created with value k=1. Output at the next page:

Classification Algorithms Report



It's clearly visible that model is very accurate. Only 1 value from class 0 is wrongly defined.

```
print("Final Model Accuracy Score")
print(accuracy_score(y_test, y_pred))
print("\nFinal Model Classification Report")
print(classification_report(y_test, y_pred))
```

Lastly, final model accuracy and classification report is printed out.

```
Final Model Accuracy Score
0.9975728155339806

Final Model Classification Report
              precision    recall  f1-score   support

     0           1.00       1.00       1.00        229
     1           0.99       1.00       1.00        183

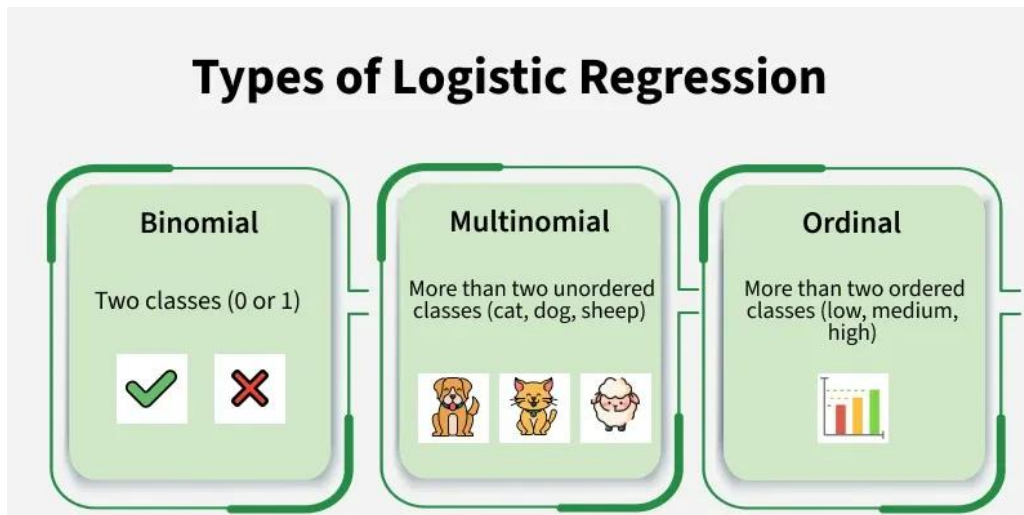
   accuracy              1.00              412
  macro avg           1.00       1.00       1.00       412
weighted avg           1.00       1.00       1.00       412
```

2.Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression, which predicts continuous values it predicts the probability that an input belongs to a specific class.

It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1.

It uses sigmoid function to convert inputs into a probability value between 0 and 1.

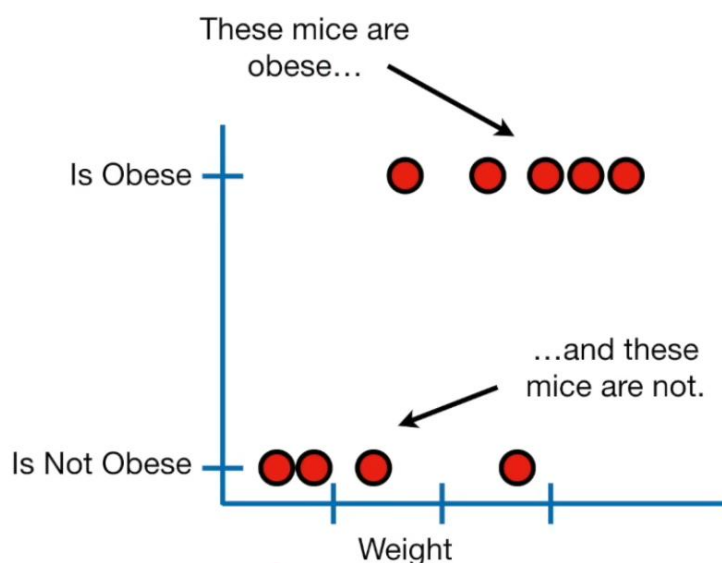


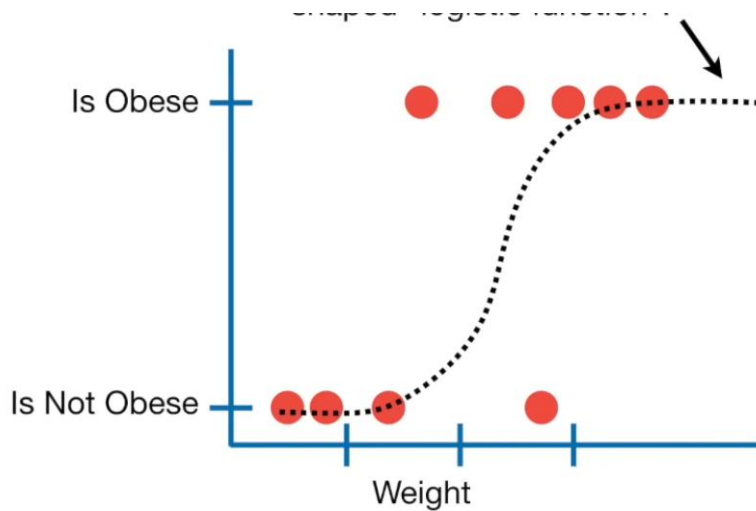
In Logistic Regression we must assume that each point is independent from other points.

There must be no extreme outliers in the dataset.

Larger dataset allows for more stable predictions.

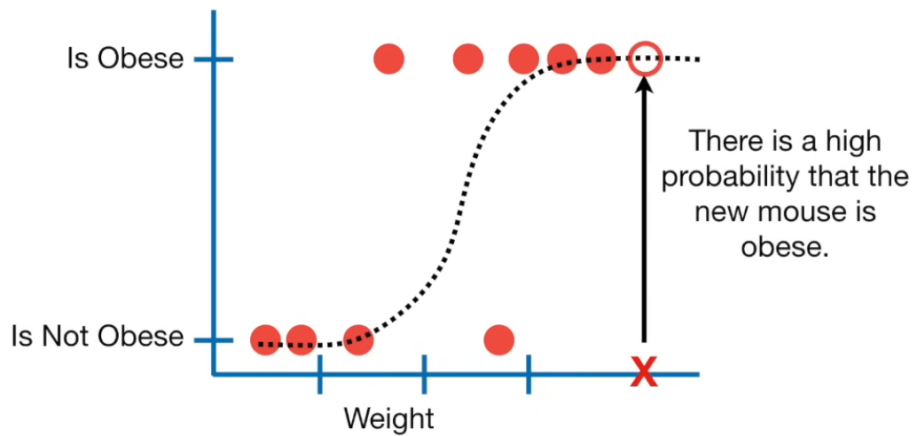
Here is a logistic regression example that I have taken from youtube video **StatQuest: Logistic Regression** from StatQuest.



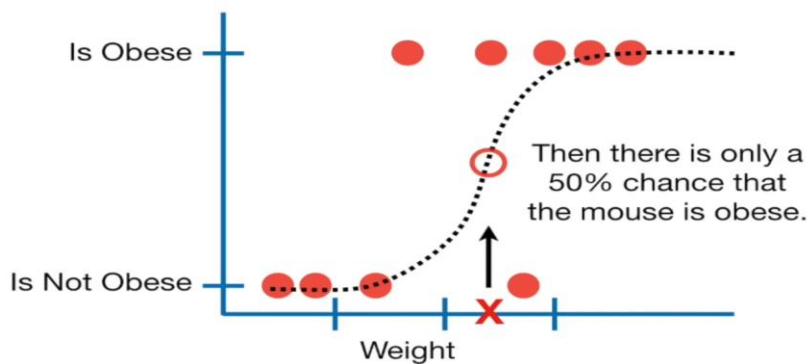


Here, we can predict that mouse is obese or not by looking at its weight.

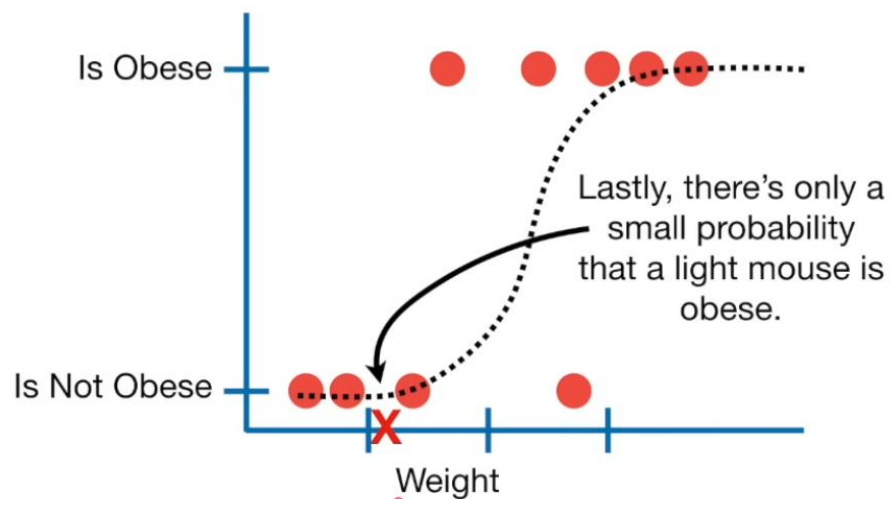
If we found very fat mouse, it can be predicted that it is obese.



If we found intermediate weight mouse it can obese or not depends on the minimum threshold of the graph.



Here, weight is at minimum threshold. If it is bigger than the limit, it is predicted as obese mouse.



In this report Binomial and Multinomial Logistic Regression will be explained.

2.1 Binomial Logistic Regression

In binomial logistic regression, the target variable can only have two possible values such as "0" or "1", "pass" or "fail". The sigmoid function is used for prediction.

We will be using scikit-learn library for this and shows how to use the breast cancer dataset to implement a Logistic Regression model for classification.

Here are all the libraries that are implemented.

```
# Binomial Logistic Regression
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
x, y = load_breast_cancer(return_X_y=True)

x_train, x_test, y_train, y_test = train_test_split(*arrays: x, y, test_size=0.2, random_state=23)
clf = LogisticRegression(max_iter=10000, random_state=0)
# by default min threshold is 0.5
clf.fit(x_train, y_train)
```

Here, we defined features and target for the dataset.

Then we split %20 of the data for testing.

By using LogisticRegression(), we created logistic regression model named clf that has maximum limit of 10000 iterations.

After that model is trained .

```
acc = accuracy_score(y_test, clf.predict(x_test)) * 100
print(f"Logistic Regression Model Accuracy: {acc:.2f}%")
```

Lastly, accuracy score percentage is printed out.

```
Logistic Regression Model Accuracy: 96.49%
```

2.2 Multinomial Logistic Regression

In multinomial logistic regression, target variable can have 3 or more possible types which are not ordered i.e types have no quantitative significance like “disease A” vs “disease B” vs “disease C”.

We will again only use scikit-learn library to implement regression.

Here are all the imported extensions.

```
from sklearn.model_selection import train_test_split
from sklearn import datasets, linear_model, metrics
```

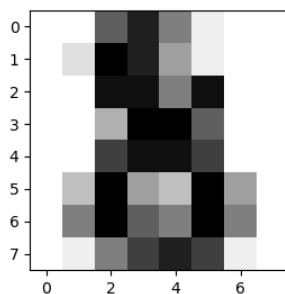
```
digits = datasets.load_digits()
X = digits.data
y = digits.target

X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.4, random_state=1)

reg = linear_model.LogisticRegression(max_iter=10000, random_state=0)
reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)
```

Digits dataset from sklearn is implemented.

This dataset is made up of 1797 8x8 images. Each image, like the one shown below, is of a hand-written digit.



Features and target is implemented.

%40 of the data is assigned for testing the dataset.

Regression model is created with maximum limit of 10000 iterations.

Model is trained and predictions are made.

```
print(f"Logistic Regression model accuracy: {metrics.accuracy_score(y_test, y_pred) * 100:.2f}%")
```

Lastly, model accuracy is printed out.

```
Logistic Regression model accuracy: 96.66%
```

2.3 Logistic Regression with Bank Marketing Dataset

For this program Bank Marketing Dataset is used. The dataset originates from direct telephone marketing campaigns conducted by a Portuguese banking institution. Each row represents a customer contacted during a marketing campaign. Here is the some part of the dataset:

	age	job	marital	education
1	58	management	married	tertiary
2	44	technician	single	secondary
3	33	entrepreneur	married	secondary
4	47	blue-collar	married	unknown
5	33	unknown	single	unknown
6	35	management	married	tertiary
7	28	management	single	tertiary
8	42	entrepreneur	divorced	tertiary

Target column can take 'yes' or 'no' so this dataset implemented to show Binomial Logistic Regression.

no
no
no
no
yes
no
no
yes
yes
no

Here are all the libraries added to the program.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

Different from previous examples, OneHotEncoder from sklearn is added.

One-Hot Encoding is a data preprocessing technique used to convert categorical data into a numerical format.

```
pd.set_option(*args: 'display.max_columns', 20)
pd.set_option(*args: 'display.max_rows', 100)
pd.set_option(*args: 'display.max_colwidth', 100)
pd.set_option(*args: 'display.width', 300)

df = pd.read_csv(filepath_or_buffer: 'bank-full.csv', sep=";")

print("\nFirst 5 Rows of the Bank Dataset")
print(df.head())

print("\nLast 5 Rows of the Bank Dataset")
print(df.tail())

print("\nBank Dataset Description")
print(df.describe())

print("\nBank Dataset Information")
print(df.info())

print("\nBank Dataset Shape Information")
print(df.shape)

print("\nMissing Values in the Bank Dataset")
print(df.isnull().sum())
print("There are no missing values in the Bank Dataset\n")
```

Dataset information is printed out.

```
df_for_regression = df.drop(labels: 'y', axis=1)
df_for_regression['default'] = df_for_regression['default'].map({"yes":1,"no":0})
df_for_regression['housing'] = df_for_regression['housing'].map({"yes":1,"no":0})
df_for_regression['loan'] = df_for_regression['loan'].map({"yes":1,"no":0})
```

Here, class column is dropped for regression because this df_for_regression dataset is represent features.

Some of the binary categorical columns such as default, housing and loan which has only 'yes' and 'no' as values are changed to 1 or 0.

```
print("Bank Dataset after converting values into binary\n")
print(df_for_regression.head())

categorical_columns = ["job", "marital", "education", "contact", "month", "poutcome"]
numerical_columns = ["age", "default", "balance", "housing", "loan", "day", "duration",
                     "campaign", "pdays", "previous"]
```

Dataset for regression features are printed out.

Categorical and numerical columns are divided.

```
encoder = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
encoded_data = encoder.fit_transform(df_for_regression[categorical_columns])
encoded_df = pd.DataFrame(encoded_data, columns=encoder.get_feature_names_out(categorical_columns))
```

OneHotEncoder() function is used to convert categorical columns into numerical values.

```
y = df["y"].map({"yes": 1, "no": 0})

numerical_df = df_for_regression[numerical_columns].reset_index(drop=True)
encoded_df = encoded_df.reset_index(drop=True)

X = pd.concat(objs=[numerical_df, encoded_df], axis=1)
```

Target is defined as y and column y values which is 'yes' and 'no' converted to 1 and 0.

Then numerical columns in the features and encoded categorical values are merged to create X variable which we will use in the train_test_split as features.

```
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.3, random_state=23, stratify=y)

X_train = X_train.copy()
X_test = X_test.copy()

scaler = StandardScaler()

X_train[numerical_columns] = scaler.fit_transform(X_train[numerical_columns])
X_test[numerical_columns] = scaler.transform(X_test[numerical_columns])
```

%30 of the dataset used for testing and %30 for training.

X_train and X_test is duplicated.

StandartScaler() is implemented to standardize the dataset.

The scaler was fitted only on the training data, where it calculated the mean and standard deviation of each numerical feature. The training values were then transformed using these statistics. The test data was standardized using the same mean and standard deviation obtained from the training data.

```
model = LogisticRegression(max_iter=20000, class_weight="balanced", random_state=0)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

Regression model is created with maximum limit of 20000 iterations.

Since the target classes were imbalanced, `class_weight="balanced"` was used to assign greater importance to the minority class during training.

Model is trained and predictions happened.

```
acc = accuracy_score(y_test, y_pred) * 100
print(f"\nLogistic Regression model accuracy: {acc:.2f}%")

print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

Lastly, accuracy score and classification report is printed out.

```
Logistic Regression model accuracy: 85.28%

Classification Report
```

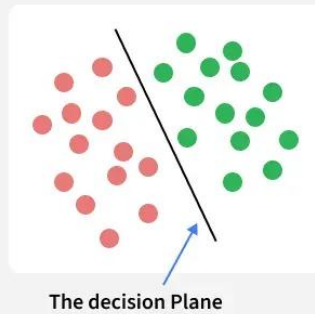
	precision	recall	f1-score	support
0	0.97	0.86	0.91	11977
1	0.43	0.81	0.56	1587
accuracy			0.85	13564
macro avg	0.70	0.84	0.74	13564
weighted avg	0.91	0.85	0.87	13564

In this project, Logistic Regression was implemented using the Bank Marketing dataset to predict whether a customer would subscribe to a term deposit.

3.SVM (Support Vector Machine) Algorithm

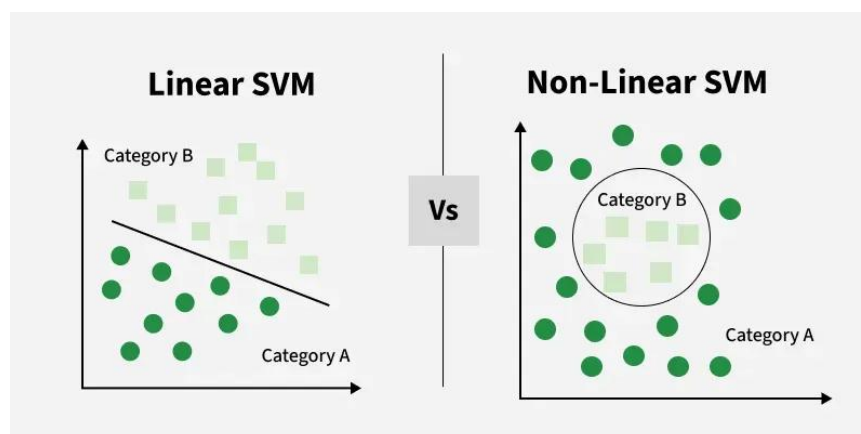
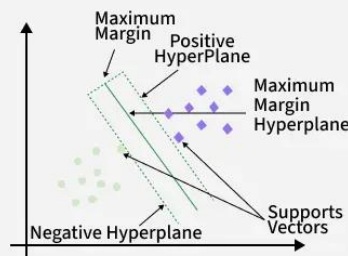
Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data. It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.

- Support Vector Machine is used for classification and regression.
- It works by finding decision planes that separate data into different classes.



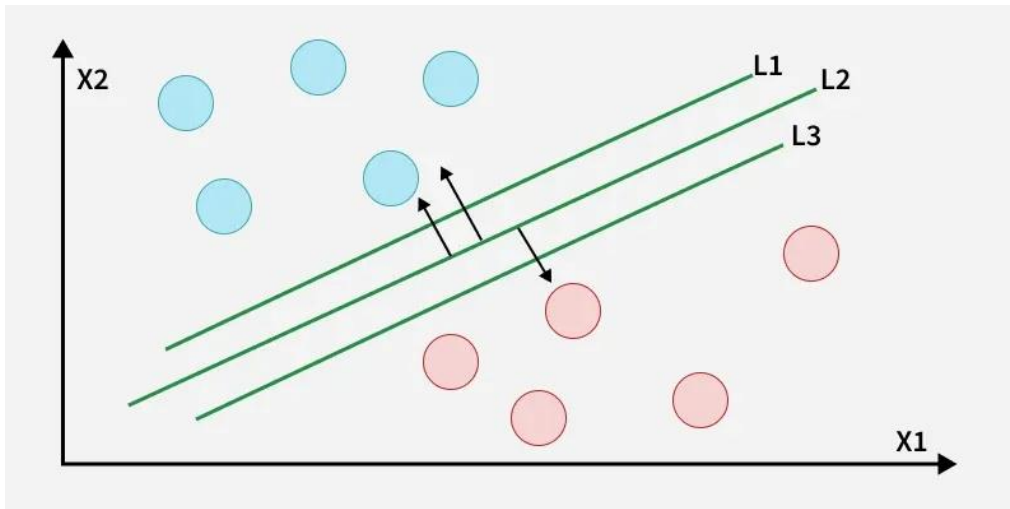
Support Vectors & Hyperplane

- Support Vectors are the closest data points to the hyperplane that define the class boundary.
- A hyperplane is a plane that separates different classes.



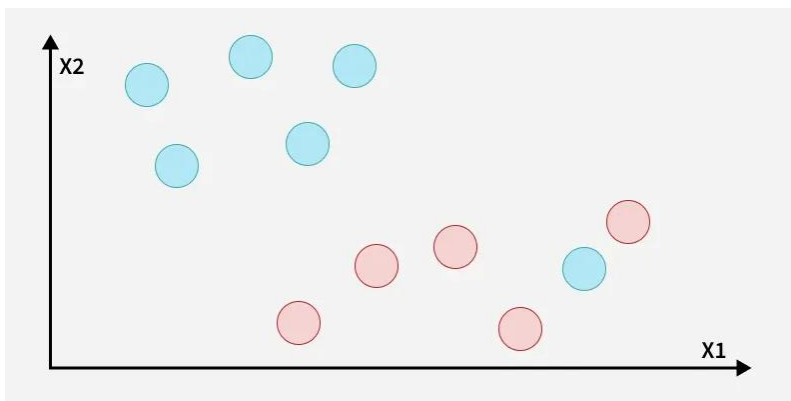
The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data.

The key idea behind the SVM algorithm is to find the hyperplane that best separates two classes by maximizing the margin between them. This margin is the distance from the hyperplane to the nearest data points (support vectors) on each side.



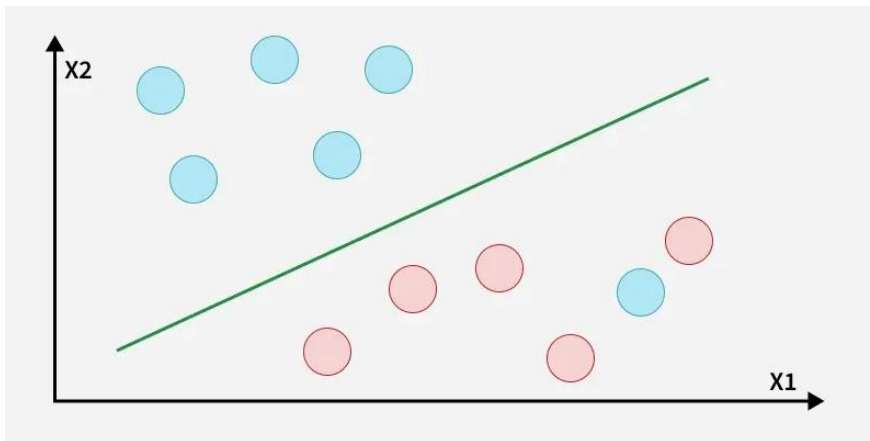
The best hyperplane also known as the hard margin is the one that maximizes the distance between the hyperplane and the nearest data points from both classes. This ensures a clear separation between the classes. So from the above figure, we choose L2 as hard margin.

Lets consider a small dataset like this:



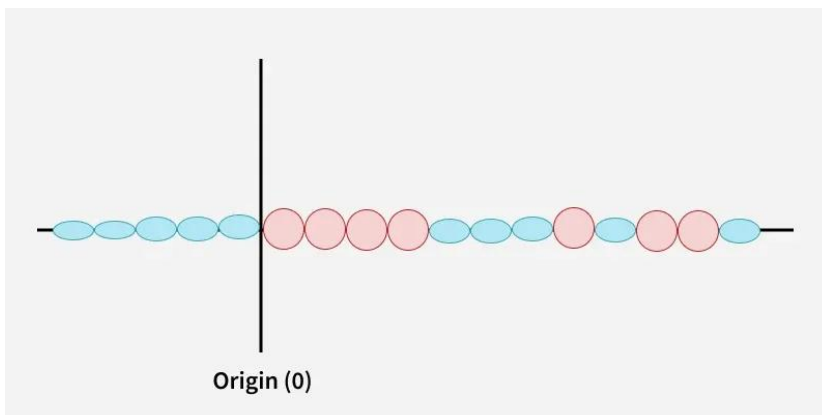
How can we classify the data?

The blue ball in the boundary of red ones is an outlier of blue balls. The SVM algorithm has the characteristics to ignore the outlier and finds the best hyperplane that maximizes the margin. SVM can be sensitive to outliers, especially in the case of a hard margin, while soft margin SVM helps reduce their impact by allowing some misclassifications.

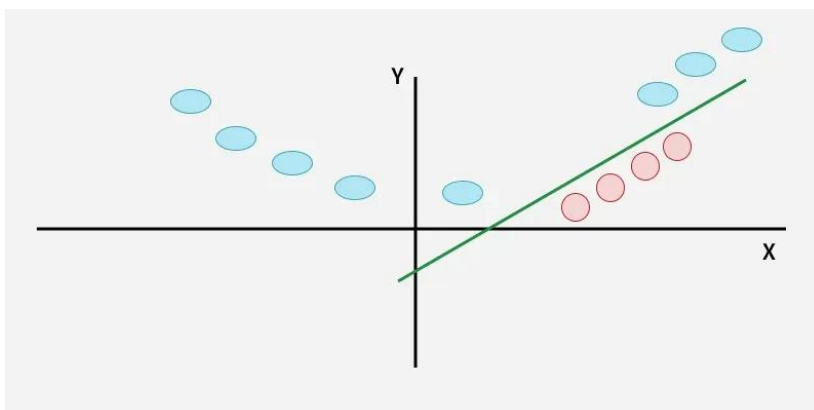


When data is not linearly separable i.e it can't be divided by a straight line, SVM uses a technique called kernels to map the data into a higher-dimensional space where it becomes separable. This transformation helps SVM find a decision boundary even for non-linear data.

Original 1D Dataset:



A kernel is a function that maps data points into a higher-dimensional space without explicitly computing the coordinates in that space. This allows SVM to work efficiently with non-linear data by implicitly performing the mapping. For example consider data points that are not linearly separable. By applying a kernel function SVM transforms the data points into a higher-dimensional space where they become linearly separable.



There are different types of kernel functions which will be later explained with examples.

3.1 Linear SVM Example with Breast Cancer Dataset

We will predict whether cancer is Benign or Malignant using historical data about patients diagnosed with cancer. This data includes independent attributes such as tumor size, texture, and others. To perform this classification, we will use an SVM (Support Vector Machine) classifier to differentiate between benign and malignant cases effectively.

Here are all the libraries that implemented:

```
from sklearn.svm import SVC
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.inspection import DecisionBoundaryDisplay
```

Matplotlib added for visualizing the SVM.

Scikit-learn used to import breast cancer dataset, add SVM Algorithm and create decision boundary display.

```
cancer = load_breast_cancer()
X = cancer.data[:, :2]
y = cancer.target
```

Here, we loaded the dataset and defined features, target.

```
svm = SVC(kernel='linear', C=1)
svm.fit(X, y)
```

We applied the SVM model as linear and trained the model.

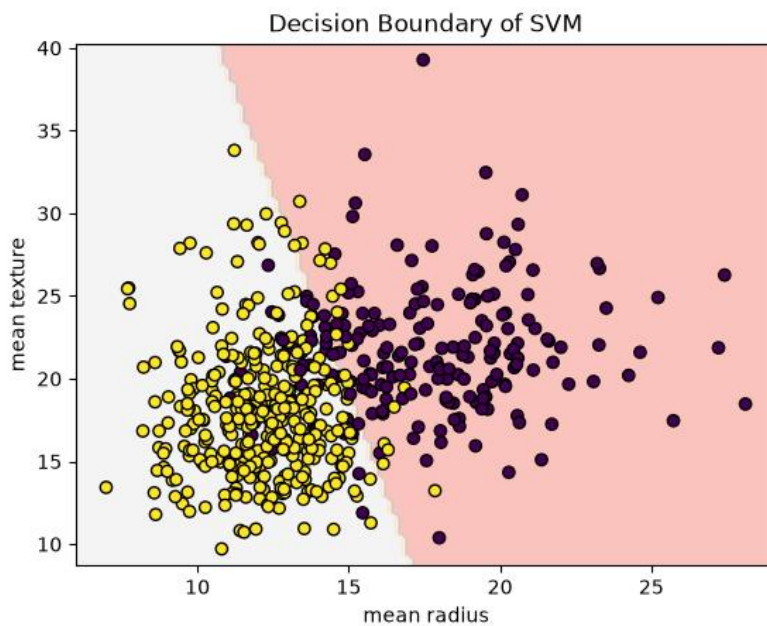
```
DecisionBoundaryDisplay.from_estimator(
    svm,
    X,
    response_method="predict",
    alpha=0.8,
    cmap="Pastel1",
    xlabel=cancer.feature_names[0],
    ylabel=cancer.feature_names[1],
)
```

`DecisionBoundaryDisplay.from_estimator()` automatically creates a grid covering the two-dimensional feature space, asks the trained model to predict the class at every grid point and colors the regions according to those predictions.

```
plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k')
plt.title('Decision Boundary of SVM')
plt.savefig('decision_boundary.png')
plt.show()
```

Lastly, we visualized the decision boundary by using matplotlib.

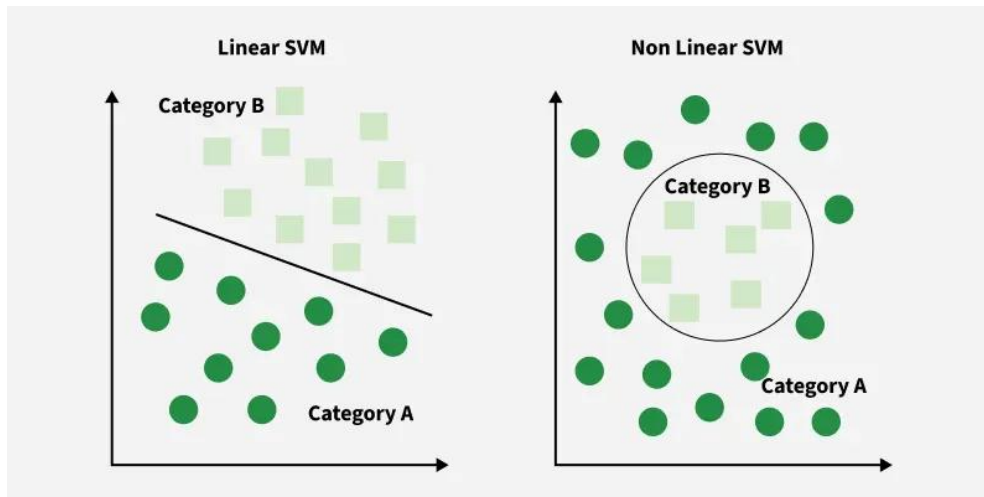
Here is the graph:



We can clearly see that yellow and purple data points are separated using boundary.

3.2 Non Linear SVM Example

Support Vector Machines are algorithms for classification and regression tasks. However, the standard (linear) SVM can only classify data that is linearly separable, meaning a straight line can separate the classes (in 2D) or a hyperplane (in higher dimensions). Non-linear SVM extends SVM to handle complex, non-linearly separable data using kernels.

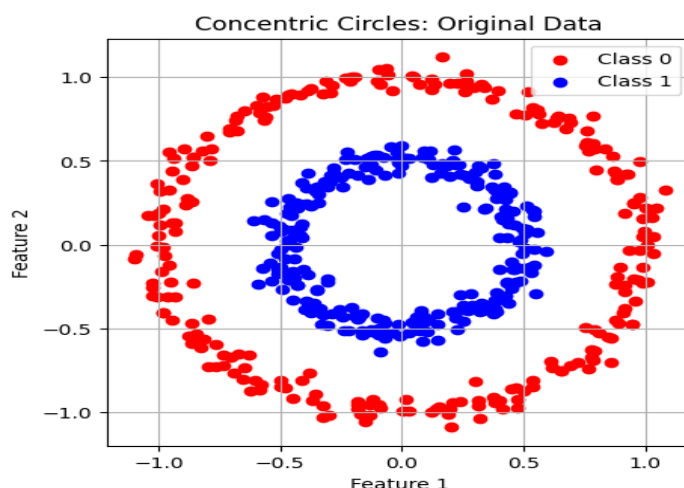


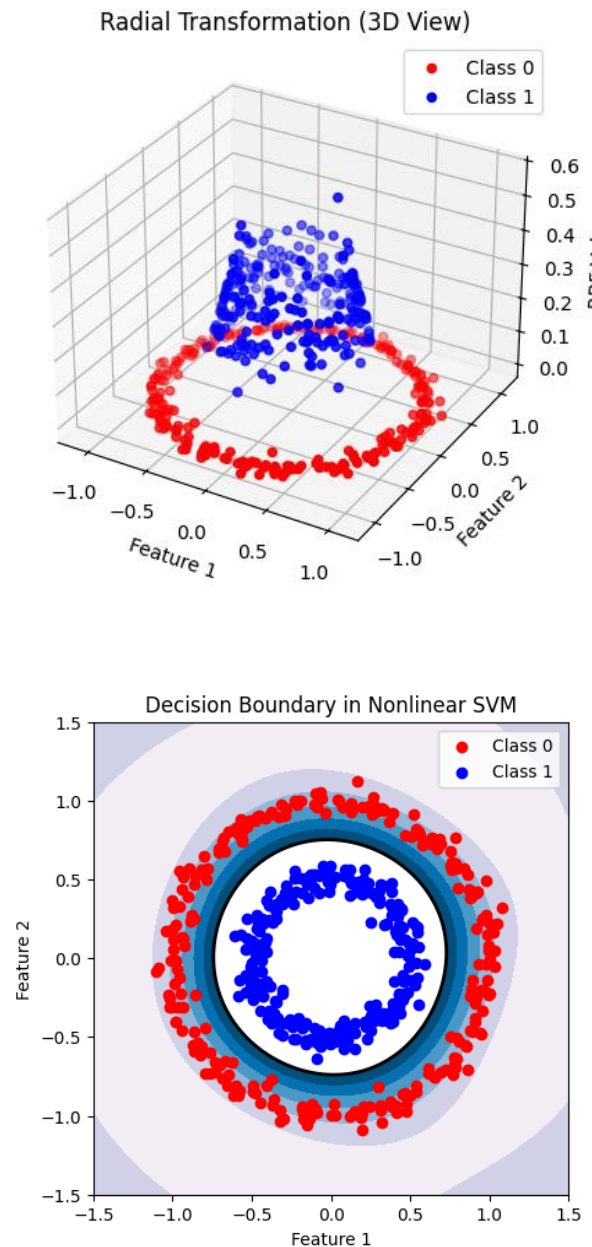
For example, imagine classifying fruits like apples and oranges based on features like colour and texture. The apple data points might form a circular cluster surrounded by oranges. A simple SVM can't separate them, but a non-linear SVM handles this by using kernel functions to create curved boundaries, allowing it to classify such complex, non-linear patterns accurately.

Instead of explicitly transforming data, the kernel computes dot products in a higher-dimensional space, helping the model find patterns and separate complex data more easily.

For example, suppose we have data points shaped like two concentric circles.

If we try to separate these classes with a straight line it can't be done because the data is not linearly separable in its current form.





When we use a kernel function it transforms the original 2D data like the concentric circles into a higher-dimensional space where the data becomes linearly separable. In that higher-dimensional space the SVM finds a simple straight-line decision boundary to separate the classes.

When we bring this straight-line decision boundary back to the original 2D space it no longer looks like a straight line. Instead, it appears as a circular boundary that perfectly separates the two classes. This happens because the kernel trick allows the SVM to "see" the data in a new way enabling it to draw a boundary that fits the original shape of the data.

Now we will implement Non Linear SVM example in Python.

Here are all the libraries used for the program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
```

From scikit-learn make_circles added to create dataset that has circle shape.

Many other sklearn extensions added that we used in previous examples so no further explanation is needed.

```
X, y = make_circles(n_samples=500, factor=0.5, noise=0.05, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.3, random_state=42)
```

Here, we created the dataset with 500 data points. Factor controls the size of the inner circle relative to the outer circle. Noise adds small random changes to the coordinates, making the data more realistic and less perfectly shaped.

After that, train_test_split is applied to X and y which is target and features.

```
svm = SVC(kernel='rbf', C=1, gamma=0.5)
svm.fit(X_train, y_train)
y_pred = svm.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
Classification_Report = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy:.2f}")
print(f"\nClassification Report: {Classification_Report}")
```

Non Linear SVM Model is created with kernel 'rbf'. Rbf is Radial Basis Function kernel. The RBF kernel enables the SVM to learn nonlinear decision boundaries.

Model is trained and predictions happened.

After that, accuracy and classification report is defined then printed out.

```
def plot_decision_boundary(X : (_getitem_), y, model : {predict}): 1 usage
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    xx, yy = np.meshgrid(np.arange(x_min, x_max, step: 0.01), np.arange(y_min, y_max, step: 0.01))

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(*args: xx, yy, Z, alpha=0.8, cmap=plt.cm.Paired)
    plt.scatter(X[:,0], X[:,1], c=y, edgecolors='k', cmap=plt.cm.Paired)
    plt.title("Non-linear SVM with RBF Kernel")
    plt.savefig("non_linear_svm_RBF.png")
    plt.show()

plot_decision_boundary(X, y, svm)
```

Here, we created the decision boundary.

First, we defined the minimum and maximum values for graph limit.

By using Numpy np, mesh grid is created.

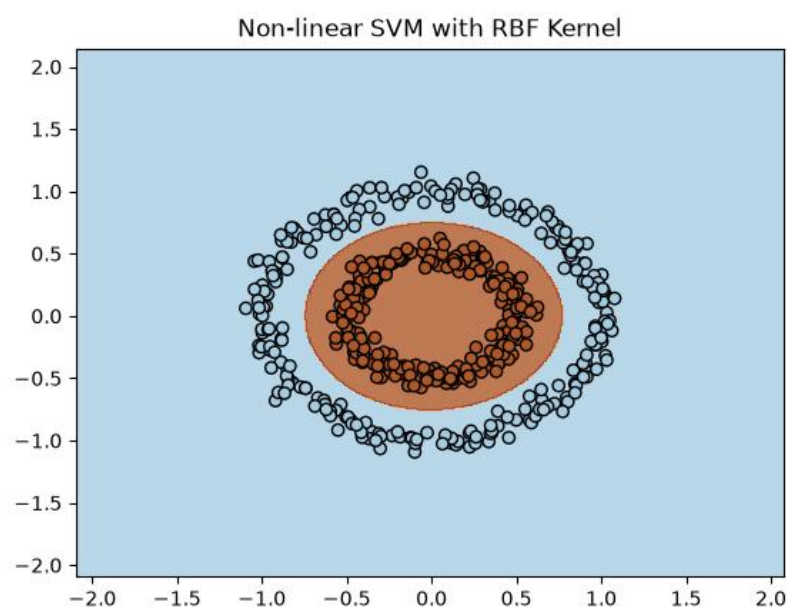
Model predicted grid points and ravel() used for flattening the two coordinate grids.

np.c_[] combines them into coordinate pairs.

Plt.contourf used to creating boundary limit.

After that, we scattered the points with each class having different colors.

Here is the graph:



As it can be seen from the graph, both circles have their boundary limit.

3.2.1 Another Non Linear SVM Example

This program is mostly the same as the previous Non Linear SVM program.

Here is the full code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

X, y = make_moons(n_samples=500, noise=0.1, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

svm_poly = SVC(kernel='poly', degree=3, C=1.0, coef0=1)
svm_poly.fit(X_train, y_train)
y_pred = svm_poly.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
classification_report = classification_report(y_test, y_pred)
print(f"\nClassification report: {classification_report}")

def plot_decision_boundary(X, y, model):
    """ 1 usage
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, step=0.01),
                        np.arange(y_min, y_max, step=0.01))

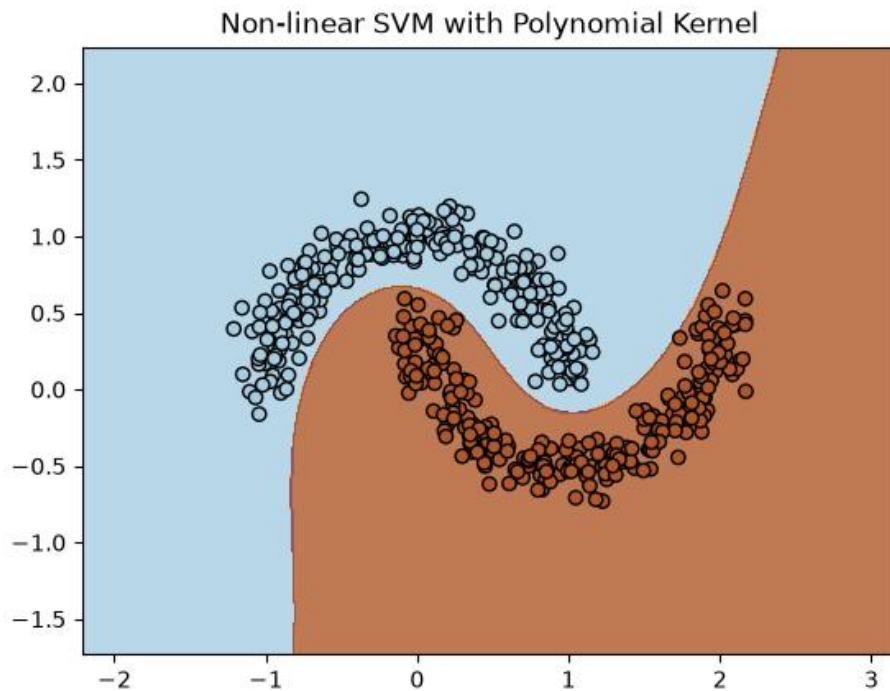
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.Paired)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolor='k', cmap=plt.cm.Paired)
    plt.title("Non-linear SVM with Polynomial Kernel")
    plt.savefig("non_linear_svm.png")
    plt.show()

plot_decision_boundary(X, y, svm_poly)
```

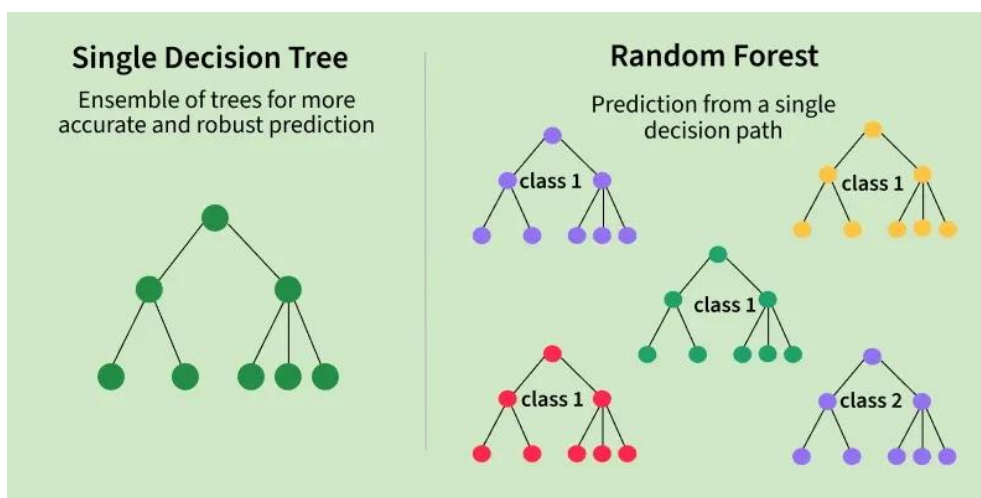
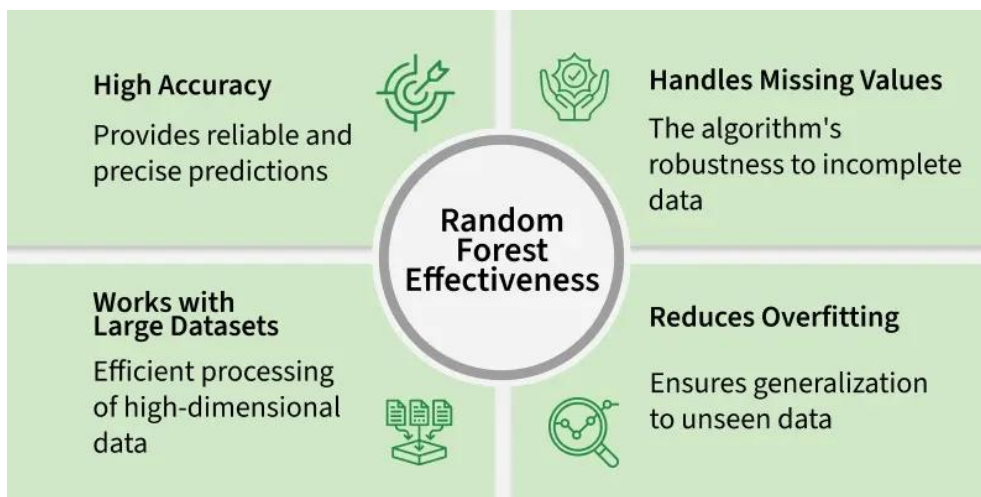
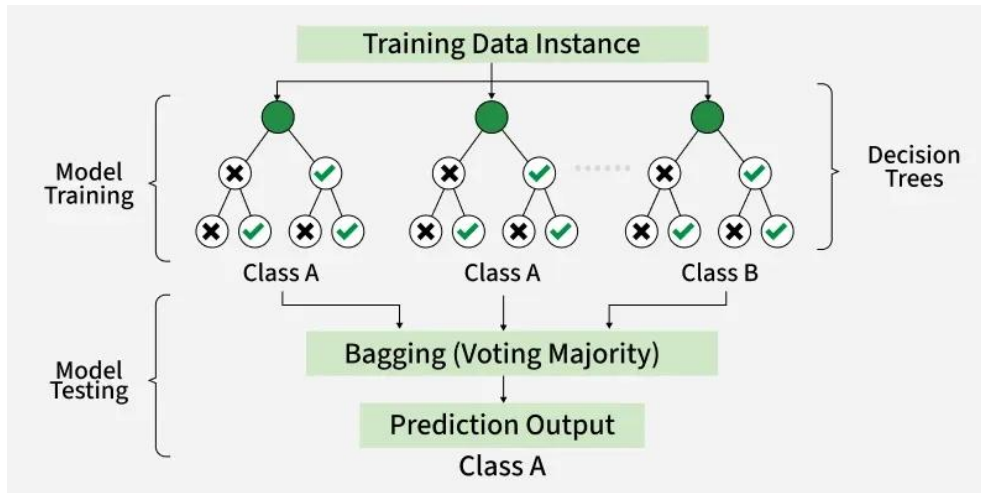

The main difference between this program and the previous nonlinear SVM example is the dataset and kernel function used. The previous example used the `make_circles` dataset, which contains an inner and an outer circular class, while this example uses the `make_moons` dataset, which consists of two interlocking crescent-shaped classes. Additionally, the previous model used the RBF kernel, whereas this model uses a polynomial kernel.

Here is the output of the program:



4. Random Forest Classifier

Random Forest is a machine learning algorithm that uses many decision trees to make better predictions. Each tree looks at different random parts of the data and their results are combined by voting for classification or averaging for regression which makes it as ensemble learning technique. This helps in improving accuracy and reducing errors.

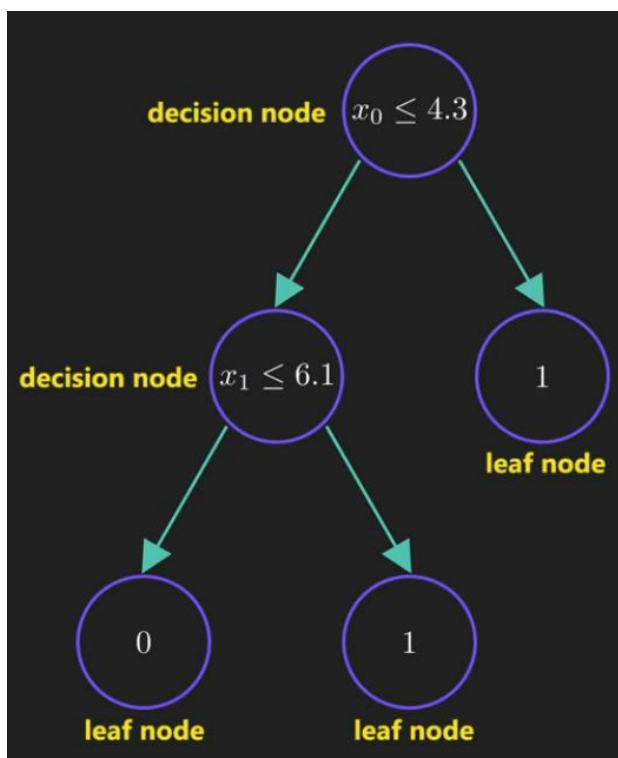


Random Forest Algorithm will be explained by using images from the video **Random Forest Algorithm Clearly Explained!** from Normalized Nerd.

Lets assume we have dataset like this.

id	x_0	x_1	x_2	x_3	x_4	y
0	4.3	4.9	4.1	4.7	5.5	0
1	3.9	6.1	5.9	5.5	5.9	0
2	2.7	4.8	4.1	5.0	5.6	0
3	6.6	4.4	4.5	3.9	5.9	1
4	6.5	2.9	4.7	4.6	6.1	1
5	2.7	6.7	4.2	5.3	4.8	1

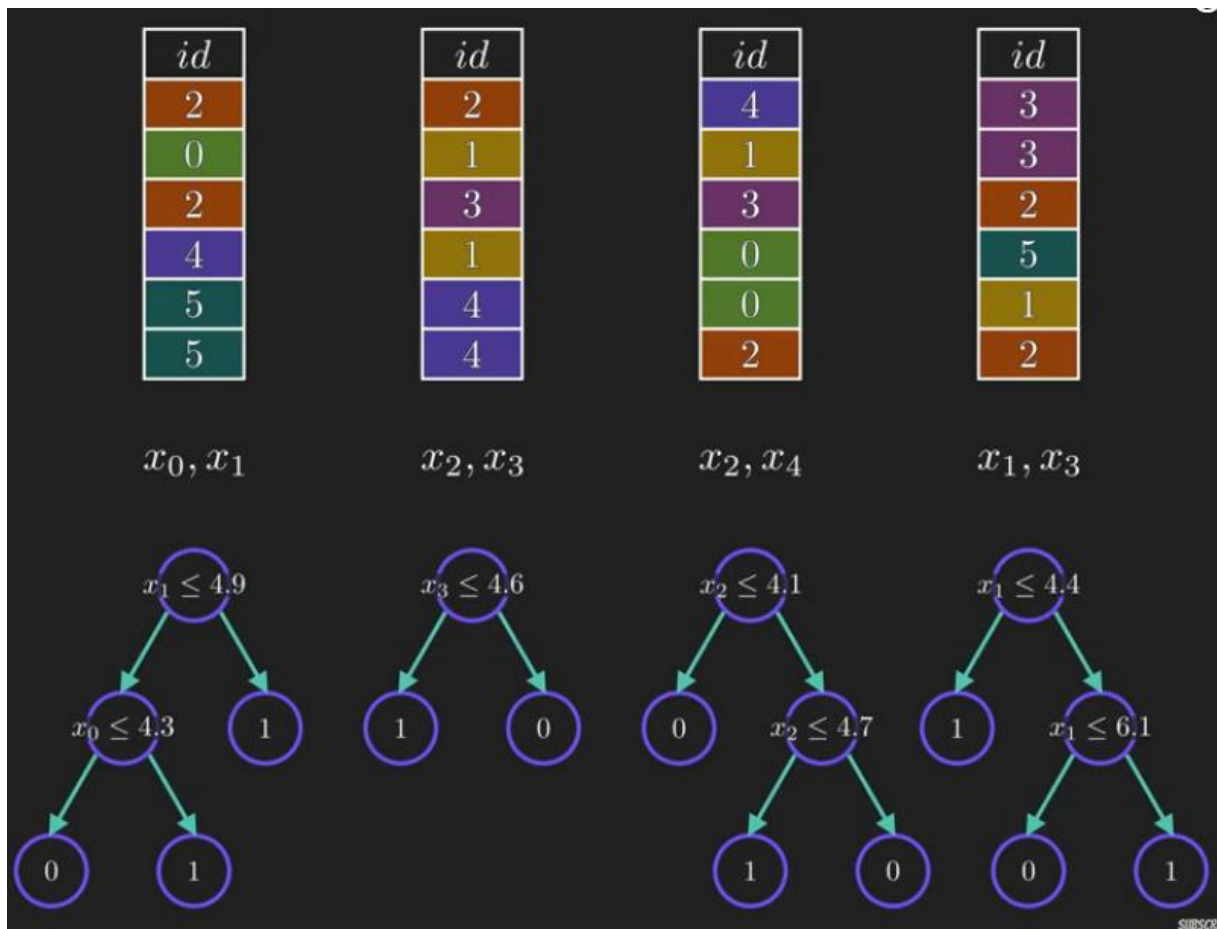
There are 5 numerical columns (features) and y column which is target.



Here is the decision tree of this dataset.

But this is only one tree and we need multiple trees.

So we take random rows from the dataset and from those random rows, we pick random columns too.



Then after picking these random rows and columns, we create decision tree of each of them.

Lets say we have a single row like this and we want to find the target value (0 or 1)

2.8	6.2	4.3	5.3	5.5
-----	-----	-----	-----	-----

We will look each decision tree that we created and take mostly voted target.

Lets say we have 10 decision trees. 7 of them says target must be 1 and the other 3 says it is 0.

Since amount of decision trees that says 1 is bigger, model will predict that row as 1.

4.1 Random Forest with Titanic Dataset

Titanic dataset has information for passengers and information that passenger survived or not. In this program, we will predict survival rate of a person in titanic.

Here are all the libraries used:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import warnings
```

From sklearn.ensemble, RandomForestClassifier is imported.

Pandas added to print dataset information and data preparation.

```
warnings.filterwarnings("ignore")

pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 350)
pd.set_option('display.max_rows', 100)
pd.set_option('display.max_colwidth', 100)

titanic = pd.read_csv('titanic.csv')

print("\nFirst 5 Rows of the Titanic Dataset")
print(titanic.head())

print("\nLast 5 Rows of the Titanic Dataset")
print(titanic.tail())

print("\nTitanic Dataset Shape")
print(titanic.shape)

print("\nTitanic Dataset Information")
print(titanic.info())

print("\nTitanic Dataset Description")
print(titanic.describe())

print("\nTitanic Dataset Missing Values")
print(titanic.isnull().sum())
```

Dataset information is printed out.

```
titanic = titanic.dropna(subset=['Survived'])

X = titanic[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']]
y = titanic['Survived']

X['Sex'] = X['Sex'].map({'female':0, 'male':1})
X['Age'] = X['Age'].fillna(X['Age'].median())
```

We dropped the values where 'Survived' target column is empty.

X and y defined. Y is defined as 'Survived' column which is target.

In the features, 'Sex' column has 2 variables: male and female. Since it has only 2 variables, we changed these variable as 1 or 0 (binary).

Rows where 'Age' column is empty, we filled it with median of the all 'Age' column values.

```
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.2, random_state=42)

rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train, y_train)

y_pred = rf_classifier.predict(X_test)
```

Here, we divided the dataset. %20 of the data for testing other %80 for training.

Model is created with 100 trees. Model is trained the predictions implemented.

```
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)

print("\nAccuracy Score")
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report")
print(classification_rep)
```

Accuracy score and classification report is printed out. Here is the output:

```
Accuracy Score
Accuracy: 0.80

Classification Report
              precision    recall  f1-score   support

     0       0.82      0.85      0.83       105
     1       0.77      0.73      0.75        74

   accuracy          0.80
  macro avg       0.79      0.79      0.79
weighted avg       0.80      0.80      0.80
```



```
sample = X_test.iloc[0:1]
prediction = rf_classifier.predict(sample)

sample_dict = sample.iloc[0].to_dict()
print(f"\nSample Passenger : {sample_dict}")
print(f"Predicted Survival: {'Survived' if prediction[0] == 1 else 'Did Not Survive'}")
```

Lastly, we took a passenger from the dataset to find passenger survived or not.

```
Sample Passenger : {'Pclass': 3.0, 'Sex': 1.0, 'Age': 28.0, 'SibSp': 1.0, 'Parch': 1.0, 'Fare': 15.2458}
Predicted Survival: Did Not Survive
```

4.2 Random Forest Regressor

Random Forest Regressor is a supervised machine-learning algorithm used to predict a continuous numerical value. It builds multiple decision trees and combines their outputs to create a more stable prediction.

Unlike Random Forest Classifier, which predicts a category, Random Forest Regressor predicts a number.

4.2. Random Forest Regressor Example

In this, we will do house price prediction by using California Housing Dataset.

Dataset consists of 20,640 samples and 9 features.

Here are all the libraries implemented.

```
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import warnings
```

mean_squared_error: measures the average squared prediction error.

r2_score: measures how much target variation the model explains.

```
pd.set_option('display.max_columns', 15)
pd.set_option('display.width', 300)
pd.set_option('display.max_rows', 50)
pd.set_option('display.max_colwidth', 100)
warnings.filterwarnings('ignore')

california_housing = fetch_california_housing()
california_data = pd.DataFrame(data=california_housing.data, columns=california_housing.feature_names)
california_data['MEDV'] = california_housing.target
```

California Housing Dataset converted into a DataFrame.

‘MEDV’ column in the dataset implemented as target. Other columns are features.

```
print("\nFirst 5 Rows of the Dataset")
print(california_data.head())

print("\nLast 5 Rows of the Dataset")
print(california_data.tail())

print("\nDataset Shape")
print(california_data.shape)

print("\nDataset Information")
california_data.info()

print("\nDataset Description")
print(california_data.describe())

print("\nMissing Values in the Dataset")
print(california_data.isnull().sum())
print("\n")
```

Dataset information is printed out.

```
X = california_data.drop(labels='MEDV', axis=1)
y = california_data['MEDV']
```

Here X is all the columns except ‘MEDV’ and y is ‘MEDV’ column which is target.


```
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.2, random_state=42)

rf_regressor = RandomForestRegressor(n_estimators=100, random_state=42)
rf_regressor.fit(X_train, y_train)
y_pred = rf_regressor.predict(X_test)
```

Train_test_split is applied.

Regressor model is created and trained then model make predictions.

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

single_data = X_test.iloc[0].values.reshape(1, -1)
predicted_value = rf_regressor.predict(single_data)

print(f"Predicted Value: {predicted_value[0]:.2f}")
print(f"Actual Value: {y_test.iloc[0]:.2f}")

print(f"Mean Squared Error: {mse:.2f}")
print(f"R-squared Score: {r2:.2f}")
```

Here, we compared our predicted value and actual value. Then mean squared error and R-squared score is printed out. Here is the output.

```
Predicted Value: 0.51
Actual Value: 0.48
Mean Squared Error: 0.26
R-squared Score: 0.80
```

It can clearly seen that model is very accurate.

4.3 Random Forest with Dry Bean Dataset

In this program, Dry Bean Dataset is used. It contains measurements obtained from images of dry bean grains. Each row represents one bean sample. It has 13,611 bean samples and 7 bean classes.

Here is the libraries of the program we implemented with Bean Dataset.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 350)
pd.set_option('display.max_rows', 150)
pd.set_option('display.max_colwidth', 100)

beans = pd.read_excel('Dry_Bean_Dataset.xlsx')

print("\nFirst 5 rows of the dataset")
print(beans.head())

print("\nLast 5 rows of the dataset")
print(beans.tail())

print("\nBeans Dataset Shape")
print(beans.shape)

print("\nBeans Dataset Information")
beans.info()

print("\nBeans Dataset Description")
print(beans.describe())

print("\nMissing values in the dataset")
print(beans.isnull().sum())
```

```
print("\nAll class variables")
print(beans['Class'].unique().tolist())
```

```
X = beans.drop(labels='Class', axis=1)
y = beans['Class']

X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.3, random_state=42)

clf = RandomForestClassifier()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)

print(f"\nAccuracy: {accuracy:.2f}")
print("\nClassification Report")
print(classification_rep)
```

5. Naive Bayes Classifier

Naive Bayes is a machine learning classification algorithm that predicts the category of a data point using probability. It assumes that all features are independent of each other. Naive Bayes performs well in many real-world applications such as spam filtering, document categorisation and sentiment analysis.

It calculates the probability that an observation belongs to each possible class and selects the class with the highest probability.

Gaussian and Multinomial Naive Bayes are gonna explained in this report.

Gaussian Naive Bayes is used mainly with continuous numerical features. It assumes that values within each class approximately follow a normal distribution.

Multinomial Naive Bayes is commonly used with count or frequency data.

Bernoulli Naive Bayes is used with binary features.

Reason it called 'Naive' is Naive Bayes assumes that all features are conditionally independent when the class is known.

5.1 Multinomial Naive Bayes

Multinomial Naive Bayes is a variation of the Naive Bayes algorithm designed for discrete data. It is commonly used in text classification, where features represent word counts or frequencies.

Works with word frequencies by modeling how often words appear in a document.

Assumes a multinomial distribution for features like words.

Commonly used in spam detection, document classification, and sentiment analysis.

Here is a Python application that detects messages as spam or not spam.

```
import pandas as pd
from pandas._libs.tslibs import vectorized
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
```

CountVectorizer converts text messages into numerical word-count vectors.

```
pd.set_option('display.max_columns', 5)
pd.set_option('display.max_rows', 50)
pd.set_option('display.width', 250)

data = {
    'text': [
        'Free money now',
        'Call now to claim your prize',
        'Meet me at the park',
        'Let's catch up later',
        'Win a new car today!',
        'Lunch plans?',
        'Congratulations! You won a lottery',
        'Can you send me the report?',
        'Exclusive offer for you',
        'Are you coming to the meeting?'
    ],
    'label': ['spam', 'spam', 'not spam', 'not spam', 'spam', 'not spam', 'spam', 'not spam', 'spam', 'not spam']
}

df = pd.DataFrame(data)
print(df.head())
```

Here is our data with different messages and their class labels. {spam, not spam}

```
df['label'] = df['label'].map({'spam': 1, 'not spam': 0})

X = df['text']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.3, random_state=42)

vectorizer = CountVectorizer()
X_train_vectors = vectorizer.fit_transform(X_train)
X_test_vectors = vectorizer.transform(X_test)
```

This section converts the text messages into numerical word-count vectors because Multinomial Naive Bayes cannot process raw text directly.

Vectorizer applied both training and testing dataset.

```
model = MultinomialNB()
model.fit(X_train_vectors, y_train)

y_pred = model.predict(X_test_vectors)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%\n")
```

```
custom_message = ["Congratulations, you've won a free vacation"]
print(custom_message)
custom_vector = vectorizer.transform(custom_message)
prediction = model.predict(custom_vector)
print("Prediction for custom message:", "Spam" if prediction[0] == 1 else "Not Spam")

custom_message2 = ["Are you coming to lunch?"]
print(custom_message2)
custom_vector2 = vectorizer.transform(custom_message2)
prediction2 = model.predict(custom_vector2)
print("Prediction for custom message:", "Spam" if prediction2[0] == 1 else "Not Spam")

custom_message3 = ["Grüezi zäme, Ich bin Alperen."]
print(custom_message3)
custom_vector3 = vectorizer.transform(custom_message3)
prediction3 = model.predict(custom_vector3)
print("Prediction for custom message:", "Spam" if prediction3[0] == 1 else "Not Spam")
```

Here, we give different custom messages to see message is spam or not.

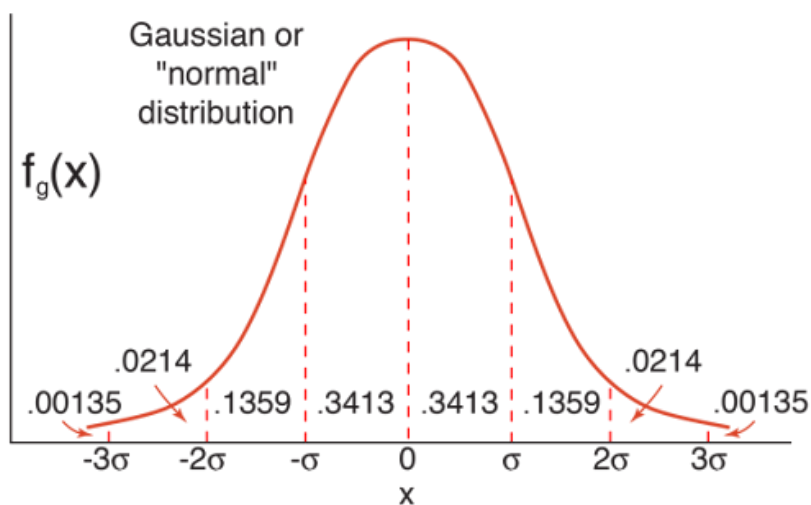
```
["Congratulations, you've won a free vacation"]  
Prediction for custom message: Spam  
['Are you coming to lunch?']  
Prediction for custom message: Not Spam  
['Grüezi zäme, Ich bin Alperen.']  
Prediction for custom message: Not Spam
```

5.2 Gaussian Naive Bayes

Gaussian Naive Bayes is a type of Naive Bayes method working on continuous attributes and the data features that follows Gaussian distribution throughout the dataset. This “naive” assumption simplifies calculations and makes the model fast and efficient.

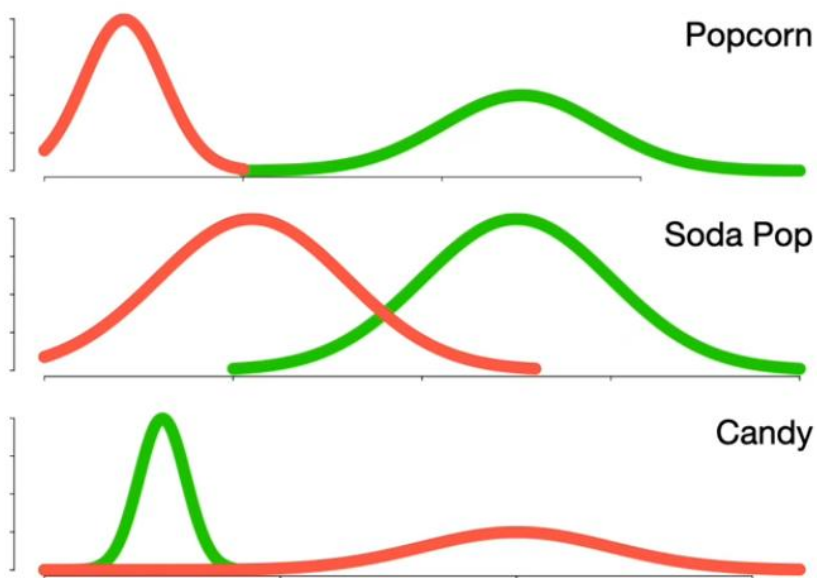
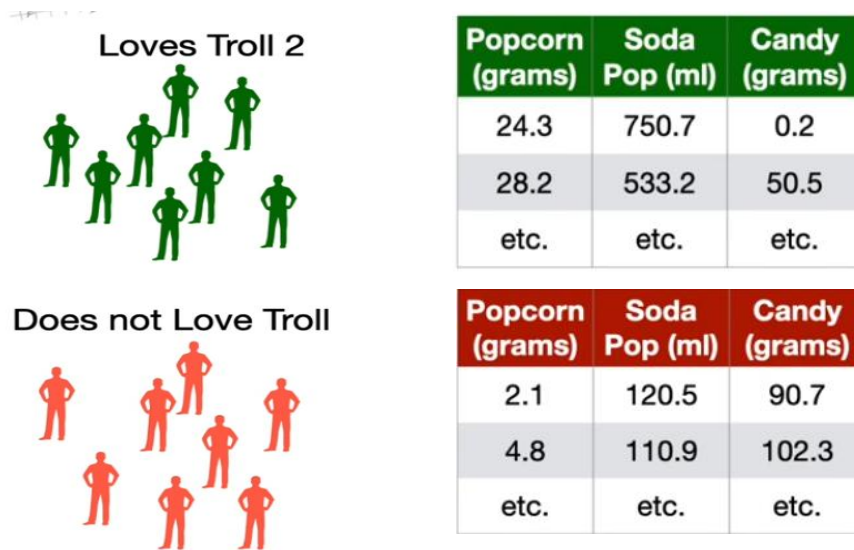
To understand Gaussian Naive Bayes, Gaussian Distribution must known.

The Gaussian distribution, also called the normal distribution, is a probability distribution in which values are concentrated around an average and become less common as they move farther away from it.



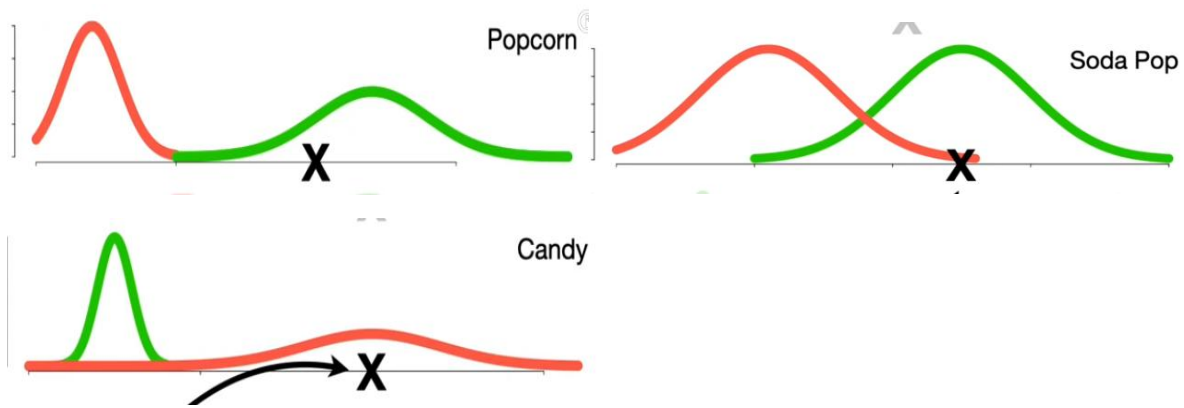
I will explain how Gaussian Naive Bayes works with the youtube video called **Gaussian Naive Bayes, Clearly Explained!!!** from StatQuest with Josh Starmer.

Lets assume we have 2 different classes. One that loves Troll 2 movie and one dont.



Here, we took out each classes information (popcorn, soda pop, candy) and draw their gaussian distribution.

Classification Algorithms Report



We take each y value for all distributions and multiply them with each other.

If we take first class (loves troll 2), we must also add percentage of it to multiplication.

For second class (doesn't love troll 2), do the same thing.

Compare both of them and choose the bigger multiplication to prediction.

Here is a Python example:

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=100, n_features=2, n_redundant=0, n_clusters_per_class=1, random_state=42)

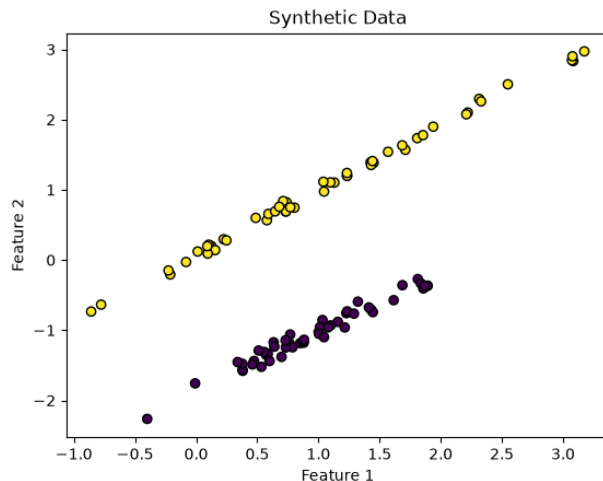
plt.scatter(X[:,0], X[:,1], c=y, cmap='viridis', edgecolors='k')
plt.title("Synthetic Data")
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.savefig('synthetic_data.png')
plt.show()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy}')
```

Synthetic dataset is created for this example.



5.3 Gaussian Naive Bayes with Real Dataset

This program uses the Adult dataset, also known as the Census Income dataset, from the UCI Machine Learning Repository.

The dataset was extracted from the 1994 United States Census database. Its objective is to predict whether an individual's annual income is:

- $\leq 50K$: \$50,000 or less
- $> 50K$: more than \$50,000

Therefore, this is a binary classification problem.

Here is the full code:

```
1 import pandas as pd
2 from sklearn.naive_bayes import GaussianNB
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score, classification_report
5 from sklearn.preprocessing import LabelEncoder
6
7 pd.set_option('display.max_columns', 20)
8 pd.set_option('display.max_rows', 100)
9 pd.set_option('display.width', 350)
10 pd.set_option('display.max_colwidth', 100)
11
12 url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
13
14 column_names = ['age', 'workclass', 'fnlwgt', 'education', 'education-num',
15                 'marital-status', 'occupation', 'relationship', 'race',
16                 'sex', 'capital-gain', 'capital-loss', 'hours-per-week',
17                 'native-country', 'income']
```

Classification Algorithms Report

```
census_data = pd.read_csv(url, names=column_names)

print("\nFirst 5 rows of the Dataset")
print(census_data.head())

print("\nLast 5 rows of the Dataset")
print(census_data.tail())

print("\nDataset Shape")
print(census_data.shape)

print("\nDataset Information")
print(census_data.info())

print("\nDataset Description")
print(census_data.describe())

print("\nMissing values in the Dataset")
print(census_data.isnull().sum())
print("\n")

le = LabelEncoder()
```

```
categorical_features = ['workclass', 'education', 'marital-status', 'occupation', 'relationship',
                        'race', 'sex', 'native-country', 'income']

for feature in categorical_features:
    census_data[feature] = le.fit_transform(census_data[feature])

census_data[['age', 'fnlwgt', 'education-num',
              'capital-gain', 'capital-loss', 'hours-per-week']] = census_data[['age', 'fnlwgt', 'education-num',
              'capital-gain', 'capital-loss', 'hours-per-week']].apply(lambda x: (_sub_, min, max) : (x-x.min()) / (x.max() - x.min()))
```

```
print(census_data.head())
print("\n")

X = census_data.drop(labels='income', axis=1)
y = census_data['income']

X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.3, random_state=42)

gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy}')

print("\nClassification Report")
print(classification_report(y_test, y_pred))
```

Classification Algorithms Report

Label encoding was used to convert categorical text values into numerical values because the Gaussian Naive Bayes implementation cannot directly process string data.

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country	income
0	0.301370	7	0.044302	9	0.800000	4	1	1	4	1	0.02174	0.0	0.397959	39	0
1	0.452055	6	0.048238	9	0.800000	2	4	0	4	1	0.00000	0.0	0.122449	39	0
2	0.287671	4	0.138113	11	0.533333	0	6	1	4	1	0.00000	0.0	0.397959	39	0
3	0.493151	4	0.151068	1	0.400000	2	6	0	2	1	0.00000	0.0	0.397959	39	0
4	0.150685	4	0.221488	9	0.800000	2	10	5	2	0	0.00000	0.0	0.397959	5	0

Accuracy: 0.8086805200122837

Classification Report				
	precision	recall	f1-score	support
0	0.82	0.95	0.88	7455
1	0.69	0.35	0.46	2314
accuracy			0.81	9769
macro avg	0.76	0.65	0.67	9769
weighted avg	0.79	0.81	0.78	9769

Here is the encoded dataset, accuracy score and classification report.

Conclusion

In this study, several supervised machine-learning algorithms were implemented for classification problems, including K-Nearest Neighbors, Logistic Regression, Support Vector Machine, Random Forest Classifier, Multinomial Naive Bayes and Gaussian Naive Bayes. Both artificial datasets and real-world datasets were used to examine how these algorithms operate under different data structures and preprocessing requirements.

The implementations demonstrated that preprocessing must be selected according to the algorithm and feature types. Feature standardization was especially important for distance- and margin-based algorithms such as KNN and SVM. Logistic Regression required numerical inputs, so binary variables were mapped to zero and one, while categorical variables were transformed through encoding. Random Forest did not require feature scaling because it classified observations using decision-tree thresholds. For text classification, CountVectorizer converted messages into word-count features suitable for Multinomial Naive Bayes. Gaussian Naive Bayes was used with numerical features and modelled their class-specific distributions.

The models were evaluated using accuracy, precision, recall and F1-score, while confusion matrices were used where appropriate to examine individual classification errors. The experiments showed that accuracy should not be considered alone, especially for imbalanced datasets. Recall and F1-score for minority classes were also necessary to determine whether a model identified less frequent but important outcomes.

Overall, no single classification algorithm was best for every problem. Model suitability depended on the dataset's size, class distribution, feature types, scaling requirements and degree of nonlinearity.